

Metody Programowania

Zadania do treningu

Data aktualizacji: 2026-06-18

Tablica zawartości

I	Przedmowa	3
I.1	Format zawartości	3
II	Wiązanie zmiennych	4
III	Typowanie wyrażeń	5
IV	Rekurencja ogonowa	6
V	Identyfikacja funkcji	7
VI	Funkcje na listach	8
VII	Gramatyki	10
VII.1	Zadanie 1.	10
VII.2	Zadanie 2.	12
VIII	Abstrakcyjne typy danych	14
VIII.1	Rozwinięcie typów języka	14
VIII.1.1	Język arytmetycznych wyrażeń	14
VIII.1.2	Język calc_let	15
VIII.2	Monady	16
IX	Indukcja strukturalna	18
IX.1	Formułowanie zasady indukcji	18

I. *Przedmowa*

Materiału do przedmiotu *Metod Programowania* nie da się nauczyć w tydzień. Nawet dwa tygodnie nie wystarczą. Najambitniejsi być może zdołają to uczynić w 2 tygodnie i 42 sekundy. Przedmiot ten wymaga systematycznej pracy, programowania w OCamlu na co dzień, realizowania zadań z list i pracowni, zadawania pytań. Zainteresowanie tematem języków programowania może usprawnić systematyczną pracę, pozwolić głębiej zrozumieć problemy zadawane na zajęciach.

Innymi słowy – jeżeli za $<$ tydzień masz egzamin i nie wiesz o czym jest ten przedmiot, to lepiej zmienić studia lub przygotowywać się do powtarzania MP. **Natomiast**, jeżeli skrupulatnie się uczyłeś, chodziłeś na wykłady i na nich uważałeś, próbowałeś podchodzić do większości zadań z list – nie masz raczej czego się bać. Ten zestaw pytań to będzie dla Ciebie jedynie powtórka z materiału całego wykładu, usystematyzowanie wiedzy, ćwiczenia mentalne. Na spokojnie można to przejrzeć dzień przed egzaminem, spróbować porozwiązywać parę zadań samodzielnie, a resztę dnia odpoczywać.

Jeśli jednak masz problemy z $>$ połową zadań, to na naukę zapewne za późno, ale warto próbować. Powodzenia!

I.1 *Format zawartości*

Każde zadanie jest wydrukowane na osobnej kartce, dając miejsce na samodzielne rozwiązanie/nie spojlerując odpowiedzi. Na stronach kolejnych są wszelkie wskazówki do zadania, które mogą lekko pomóc w jego rozwiązaniu. Wreszcie – rozwiązanie z wyjaśnieniem.

Ponownie, na naukę widząc ten materiał parę dni przed sesją, pewnie już za późno. Tutaj zakładam że większość rzeczy jest znane, tłumacząc tylko rzeczy mniej widoczne dla niewprawionego oka.

II. Wiązanie zmiennych

W poniższych wyrażeniach podkreśl wolne wystąpienia zmiennych. Dla każdego związanego wystąpienia zmiennej, narysuj strzałkę od tego wystąpienia do wystąpienia wiążącego je.

```
let f x =  
  
    let x = x  
  
    and y = x * y in  
  
f x y z
```

```
fun f x y ->  
  
    let z = x + y in  
  
    let x = y + x in  
  
fun y -> g x y z
```

```
let rec f x =  
  
    let rec g z y = j y (f z)  
  
    and j y z = g z (f y)  
  
in  
  
if g y > f x then  
  
    g x  
  
else  
  
    g y
```

```
let fun x = x * x in  
  
(fun x -> x * 5) y  
  
|> (fun x -> y)  
  
|> x
```

```
let zagadka b c f =  
  
    match b with  
  
    | [] -> []  
  
    | a :: b ->  
  
        zagadka b (f a c) f
```

```
let z = function  
  
    | [] -> 0  
  
    | _ :: d -> 1 + (z d)  
  
and f = z  
  
and g xs ys =  
  
    (f xs) * (f ys)
```

III. Typowanie wyrażeń

Dla poniższych wyrażeń w języku OCaml podaj ich (najogólniejszy) typ, lub napisz BRAK TYPU, gdy wyrażenie nie posiada typu (nie typuje się)

Lp	Wyrażenie	Typ
1.	<code>fun x -> x</code>	
2.	<code>fun x -> x *. 2</code>	
3.	<code>fun x y -> !x > (y+2,10)</code>	
4.	<code>let rec add a b = if b = 0 then a else add (a + 1) (b-1)</code>	
5.	<code>fun f x -> f (f x)</code>	
6.	<code>fun f -> (fun x -> f x x)(fun y -> f y y)</code>	
7.	<code>fun f x -> f (f x) > x</code>	
8.	<code>fun a b c f d g e -> f b d c (g a)</code>	
9.	<code>List.fold_right</code>	
10.	<code>List.map</code>	
11.	<code>List.iter2</code>	
12.	<code>fun f x y -> f</code>	

IV. Rekurencja ogonowa

Określ czy definicje poniższych funkcji rekurencyjnych są ogonowe. Jeżeli tak, wpisz w komórce obok TAK, jeżeli nie – przepisz je na ogonowe, lub uzasadnij, dlaczego to niemożliwe.

Lp	Definicja funkcji	Odpowiedź
1.	<pre>let rec f a = if a < 0 then -1 else if a = 0 then a else a + f (a-1)</pre>	
2.	<pre>let rec comp f fil a = if fil a then a else comp f fil (f a)</pre>	
3.	<pre>let rec f g xs = match xs with [] -> g 0 x :: xs -> (g x) :: (f g xs)</pre>	
4.	<pre>let rec app xs ys = match xs with [] -> ys x :: xs -> x :: (app xs ys)</pre>	
5.	<pre>let rec comp f fil a = if fil a then a else 1 + comp f fil (f a)</pre>	
6.	<pre>let rec f g xs acc = match xs with [] -> acc x :: xs -> f g xs (g acc x)</pre>	

V. Identyfikacja funkcji

W każdym rzędzie podana jest funkcja w postaci typu, definicji i jej działania. Wypełnij brakujące informacje.

Pierwszy wpis (dla `square`) jest podany jako przykład.

Lp	Typ funkcji	Definicja funkcji	Opis funkcji
1.	<code>int -> int</code>	<code>fun a -> a * a</code>	Podnosi argument a do kwadratu.
2.	<code>int -> 'a -> 'a list -> 'a list</code>		Wstawia n elementów a na początek listy xs .
3.		<code>fun a b c -> if a > 0 then b a else c a</code>	
4.	<code>(ident * expr) list -> ident -> expr -> (ident * expr) list</code>		Aktualizuje (jeżeli para z pierwszą współrzędną równą <code>ident</code> jest już w liście, to ją najpierw usuwa) listę par wprowadzając nową parę <code>(ident, expr)</code> .
5.			Zamienia listę elementów typu τ <code>opt</code> na listę elementów typu τ , pozbywając się pustych elementów.

VI. Funkcje na listach

Zaimplementuj i otypuj poniższe funkcje biblioteczne. **Nie** możesz korzystać z funkcji bibliotecznych innych niż zaimplementowane.

Podkreśl nazwy funkcji które są rekurencyjne ogonowo.

Lp	Nazwa funkcji	Typ funkcji	Definicja funkcji
1.	<code>List.fold_left</code>		
2.	<code>List.fold_right</code>		
3.	<code>List.map</code>		
4.	<code>List.filter</code>		

Zaimplementuj funkcje opisane słownie poniżej, korzystając wyłącznie z powyżej wymienionych funkcji. (Jeżeli nie zaimplementowałeś danej funkcji, ale wiesz co ona robi, nadal możesz z niej skorzystać – poprawność przekazywania argumentów nie będzie wtedy brana pod uwagę)

W szczególności nie możesz definiować kolejnych funkcji rekurencyjnych.

Lp	Opis słowny	Definicja funkcji
1.	Funkcja przyjmująca listę liczb całkowitych, licząca ich sumę.	

2.	Funkcja przyjmująca listę list liczb całkowitych, licząca maksimum sum podlist.	
3.	Funkcja przyjmująca listę list zwracająca <code>true</code> wtw. gdy długości tych podlist są w kolejności rosnącej. (np. <code>[(4);(5);(100)]</code> , gdzie <code>(x)</code> oznacza listę o długości <code>x</code> zwróci <code>true</code> , a <code>[(3);(5);(4)]</code> zwróci <code>false</code>)	
4.	Funkcja przyjmująca listę par typu <code>(('a -> 'b) * 'a list)</code> , zwracająca listę list w których każdy element został zaaplikowany do odpowiedniej funkcji w pierwszej współrzędnej pary. (np. wynikiem <code>[(fun x -> x * x), 2;3;4]</code> będzie <code>[[4;9;16]]</code>)	

VII. Gramatyki

VII.1 Zadanie 1.

Dla poniżej zapisanych gramatyk określ czy są one jednoznaczne. Jeżeli nie, napisz kontrprzykład w ramce.

1. $G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{a\},$$

$$P = \{$$

$$S \rightarrow aS,$$

$$S \rightarrow \varepsilon S,$$

$$S \rightarrow a$$

$$\}$$

2. $G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{(\,,\,)\},$$

$$P = \{$$

$$S \rightarrow SS,$$

$$S \rightarrow (S),$$

$$S \rightarrow \varepsilon$$

$$\}$$

3.

$G = (N, T, P, S)$, gdzie:

$$N = \{S, D\},$$

$$T = \{1, 2, 3, +, -\},$$

$$P = \{$$

$$D \rightarrow 1, D \rightarrow 2, D \rightarrow 3$$

$$S \rightarrow D,$$

$$S \rightarrow S + S,$$

$$S \rightarrow S - S,$$

$$S \rightarrow \varepsilon$$

$$\}$$

4.

$G = (N, T, P, S)$, gdzie:

$$N = \{S, V\},$$

$$T = \{\top, \perp, \Rightarrow, \neg, a, b, \dots, z\},$$

$$P = \{$$

$$V \rightarrow \perp, V \rightarrow \top,$$

$$V \rightarrow a, V \rightarrow b, \dots, V \rightarrow z$$

$$S \rightarrow S \Rightarrow S,$$

$$S \rightarrow \neg S,$$

$$S \rightarrow \varepsilon$$

$$\}$$

VII.2 Zadanie 2.

Dla niejednoznacznych gramatyk z zadania wyżej zapisz gramatyki równoważne, które są jednoznaczne. Tam gdzie gramatyka już była jednoznaczna, zostaw pole puste.

1. $G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{a\},$$

$$P = \{$$
$$S \rightarrow aS,$$
$$S \rightarrow \varepsilon S,$$
$$S \rightarrow a$$

$$\}$$

2. $G = (N, T, P, S)$, gdzie:

$$N = \{S\},$$

$$T = \{(\,,\,)\},$$

$$P = \{$$
$$S \rightarrow SS,$$
$$S \rightarrow (S),$$
$$S \rightarrow \varepsilon$$

$$\}$$

3.

$G = (N, T, P, S)$, gdzie:

$$N = \{S, D\},$$

$$T = \{1, 2, 3, +, -\},$$

$$P = \{$$

$$D \rightarrow 1, D \rightarrow 2, D \rightarrow 3$$

$$S \rightarrow D,$$

$$S \rightarrow S + S,$$

$$S \rightarrow S - S,$$

$$S \rightarrow \varepsilon$$

$$\}$$

4.

$G = (N, T, P, S)$, gdzie:

$$N = \{S, V\},$$

$$T = \{\top, \perp, \Rightarrow, \neg, a, b, \dots, z\},$$

$$P = \{$$

$$V \rightarrow \perp, V \rightarrow \top,$$

$$V \rightarrow a, V \rightarrow b, \dots, V \rightarrow z$$

$$S \rightarrow S \Rightarrow S,$$

$$S \rightarrow \neg S,$$

$$S \rightarrow \varepsilon$$

$$\}$$

VIII. Abstrakcyjne typy danych

VIII.1 Rozwinięcie typów języka

VIII.1.1 Język arytmetycznych wyrażeń

Rozważamy język wyrażeń arytmetycznych na liczbach całkowitych z dodawaniem, odejmowaniem, dzieleniem, mnożeniem. Rozwiń następujące typy:

`type binop =`

`type expr =`

VIII.1.2 Język *calc_let*

Rozważamy język gdzie wartości mogą wartościami algebry Boole'a, liczbami całkowitymi, parami i listami wartości. Napisz typ dla *value*:

`type value =`

Dla liczb całkowitych definiujemy operację dodawania, odejmowania, mnożenia, dzielenia. Dla takich samych typów definiujemy relacje $<$, $>$, $\geq$, $\leq$, $=$. Poza tym definiujemy następujące wyrażenia:

- związanie wyrażenia do zmiennej w wyrażeniu: `let x = e1 in e2`,
- operator `cons` do dostawienia wartości na początku listy,
- `fst` i `snd` dla list.

Dany jest następujący typ składni abstrakcyjnej:

`type bop = Mult | Div | Add | Sub`

`type var = string`

```
type expr =  
  | Var    of var  
  | Int    of int  
  | Binop  of bop * expr * expr  
  | Let    of var * expr * expr  
  | Pair   of expr * expr  
  | Fst    of expr  
  | Snd    of expr
```

Dane są następujące tokeny:

```
IDENT  
MULT DIV ADD SUB EQ  
LPAR RPAR COMMA  
FST SND  
LET IN  
EOF
```

Uzupełnij brakującą definicję gramatyki w *menhir*:

`expr:`

`| LET;`

`| e = opexpr { e }`

`;`

`opexpr:`

`| i = INT { Int i }`

`| LPAR; e = expr; RPAR { e }`

`| LPAR; e1 = expr; COMMA; e2 = expr; RPAR { Pair(e1, e2) }`

`| SND; e = expr; { Snd(e) }`

`| x = IDENT { Var x }`

`;`

VIII.2 Monady

Informacja

Monady w tak bezpośredni sposób jak poniżej się nie pojawiły na wykładzie lub ćwiczeniach, ale takie pytanie ma szansę się pojawić na egzaminie.

Czym jest monada? Najprościej wyjaśnić monadę jako sposób pisania kodu w którym mamy jakiś abstrakcyjny typ `'a monada`, funkcję `return` która zamienia typ `'a` na `'a monada`, która podnosi wartość do kontekstu monady, oraz funkcję `bind` typu `'a monada -> ('a -> 'b monada) -> 'b monada`, która wyciąga wartość z kontekstu monady i aplikuje ją na funkcję.

[Wikipedia](#)

Rozważmy monadę typu `option`, zdefiniowaną następująco:

```
type 'a option =  
  | Some of 'a  
  | None
```

Napisz funkcje `return` i `bind`.

```
let return v =
```

```
let bind m f =
```


IX. *Indukcja strukturalna*

IX.1 *Formułowanie zasady indukcji*

Rozważmy następujący typ danych reprezentujący wyrażenia złożone z zer, operacji następnika i dodawań.

```
type expr =  
  | Zero  
  | Succ of expr  
  | Add of expr * expr
```

Sformułuj zasadę indukcji dla tego typu danych.